

AI Agents Cheat Sheet

Irurueta Docs › Guides & References › AI › AI Agents

Baseline: Python 3.12 · LangChain 1.4.2 / langchain-core 1.6.5 / LangGraph 1.2.12 · Java 21 · LangChain4j 1.20.2 (agentic 1.20.2-beta30) · Spring Boot 4.1 / Spring AI 2.0.1 · A2A 1.0 · MCP 2026-07-28 · running example DocsAssistant on Ollama llama3.1:8b + nomic-embed-text. Checked 2026-09-28. Quick reference only — verify against the official docs before production use.

WORKFLOW OR AGENT?

- Workflow**: code fixes the path; model fills steps. Predictable, cheap, testable.
 - Agent**: model chooses the path at run time. Flexible, costlier, harder to test.
- Don't build an agent when...**
- one good call + retrieval already works
 - the steps are known in advance (→ workflow)
 - mistakes are costly and no human can review
 - latency/cost can't absorb a variable number of calls
 - you can't measure it yet
- Five components**
model · instructions · tools · memory · orchestration (loop, plans, stop conditions, limits)
- Agent UX**
- Sync vs async; stream steps, progress, tokens: `stream_mode= ["updates", "messages", "custom"]`
 - Show sources + coarse confidence; ask one clarifying question
 - Fail gracefully, keep partial results, never fake success
 - Proactive = suggest, never act unasked on writes

AGENT LOOP & ARCHITECTURES

- perceive → reason → act → observe ∅
(stop: no tool call · limit: steps/tokens/time)
- Reflex**: rule → action, no model; front door
 - ReAct**: think → tool, one step at a time
 - Plan-and-execute**: plan → run steps → synthesise; add re-planning
 - Query decomposition**: split → answer in parallel (Send) → combine
 - Reflection / Reflexion**: draft → critique → revise; Reflexion stores lessons
 - Deep research**: plan round → investigate || → gaps? ∅ → cited report
 - Aids**: self-consistency (majority of *n* samples), tree of thoughts (beam over thoughts)

WORKFLOW PATTERNS

- `chain: A → gate → B → C`
- `route: classify → {concept | howto | decline}`
- `parallel: {answer || screen} → join · voting`
- `orch-workers: plan subtasks → workers || → synthesise`
- `eval-opt: generate ≠ evaluate (rubric, round budget)`
- LangGraph: StateGraph, conditional edges, Send, reducers
 - LangChain4j: `sequenceBuilder · conditionalBuilder · parallelBuilder · loopBuilder · supervisorBuilder`
 - Spring AI: plain `ChatClient` calls + `.entity(...)`

TOOL-DESIGN RULES

- Narrow & typed; enums over free strings
- `response_format`: concise first, detailed on request
- Paginate/truncate and say so (`has_more`)
- Actionable errors: what was wrong + what to try
- Namespace names: `docs_search`, `github_open_issue`
- Human-readable IDs (page paths, not UUIDs)
- Descriptions are prompts — evaluate them
- Stateful tools: parameterised ops, validate in code, scoped credential, log every call
- OpenAPI → tools: allow-listed, read-only only
- Many tools: semantic / hierarchical selection, tool search
- Writes**: one repo, one permission, token from env, human approval
`@tool (LangChain) · @Tool/@P (LangChain4j) · @Tool/@ToolParam (Spring AI)`

CONTEXT ENGINEERING FOR AGENTS

- $S + T + H_n + R_n + 0 \leq W$ (8192: ≈14 concise steps)
- Shrink results first (concise, paginated)
 - Clear old tool results: `ContextEditingMiddleware + ClearToolUsesEdit`
 - Compact: `SummarizationMiddleware(trigger, keep)`
 - Scratchpad never summarised: `TodoListMiddleware`, notes in state
 - Sub-agent isolation: return a condensed result
 - Just-in-time: references in context, load sections on demand
 - LangChain4j: `TokenWindowChatMemory, summarizedContext(...)`

MEMORY TYPES & APIS

- | Type | Holds |
|----------------------|---|
| Working (short-term) | this thread's messages, results, scratchpad |
| Semantic | facts, user preferences |
| Episodic | past experiences, trajectories |
| Procedural | prompts, rules, learned instructions |
- LangGraph**: checkpoint per `thread_id` (`InMemorySaver, PostgresSaver`); store across threads (`InMemoryStore(index=..., put/search by namespace)`)
 - LangChain4j**: `ChatMemory (MessageWindowChatMemory, ChatMemoryStore, @MemoryId`
 - Spring AI**: `ChatMemory → MessageWindowChatMemory (20) → ChatMemoryRepository (JDBC...); MessageChatMemoryAdvisor + CONVERSATION_ID`
 - Privacy: minimise, per-user keys, TTLs, real deletion

PLANNING

- Sequential** (plan up front) vs **stepwise** (ReAct); mix + checkpoints
 - Re-plan: continue / replan / respond, with a budget
 - Behaviour trees**: selector (?) · sequence (→) · condition · action; tick → SUCCESS / FAILURE / RUNNING
 - Back-chaining: goal → preconditions → actions
 - Reasoning model plans, small model executes
 - Plan success ≈ p^n ($0.95^{10} \approx 0.60$)
 - Failures: planning · tool · efficiency
- Observability & learning**
- OTel GenAI spans: `invoke_agent · execute_tool · chat`
 - LangSmith / Langfuse / Phoenix; Spring AI `spring.ai.tool; shadow + canary`
 - Exemplars, Reflexion lessons → SFT / DPO / RLVR
 - Thompson sampling: $\alpha += r, \beta += 1-r$

MULTI-AGENT TOPOLOGIES

- | Topology | Shape |
|------------------|--------------------------------|
| Supervisor | one coordinator calls workers |
| Hierarchical | supervisors of supervisors |
| Handoffs / swarm | active agent transfers control |
| Democratic | peers vote or debate |
| Actor-critic | produce ≠ verify |
- Add agents for isolation, tool count, conflicting roles, parallelism, privilege separation
 - LangGraph: workers as tools · `langgraph-swarm`; LangChain4j: `supervisorBuilder()`; Spring AI: `TaskTool` (community)

MCP VS A2A

- | | MCP | A2A 1.0 |
|-----------|-----------------|---|
| Connects | agent ↔ tool | agent ↔ agent |
| Unit | tool call | task + messages + artifacts |
| Discovery | list tools | <code>/.well-known/agent-card.json</code> |
| Long work | tasks extension | states, SSE, push |
- `SendMessage · SendStreamingMessage · GetTask · CancelTask · SubscribeToTask`
- Signed Agent Cards: JWS over JCS-canonical card
 - States: `TASK_STATE_WORKING, ... _INPUT_REQUIRED, ... _COMPLETED`

HUMAN-IN-THE-LOOP

- Gate every write: approve · edit · reject · respond
 - Runtime-enforced, shows the real call, durable (checkpointed)
 - `HumanInTheLoopMiddleware(interrupt_on= {...}) → __interrupt__ → Command(resume= {"decisions": [...]}))`
 - LangGraph `interrupt(value)`; node re-runs on resume
 - Java: `humanInTheLoopBuilder()` or persisted pending action + approval endpoint
 - Escalate by confidence, sample agreement, retrieval score
- autonomy: manual → suggest → act (per action)

EVALUATION METRICS

- Component (router, tool, planner, memory) vs end-to-end
- Trajectory match: strict · unordered · subset · superset
- Tool recall $|E_nC|/|E|$ · precision $|E_nC|/|C|$ · argument accuracy · forbidden-call rate = 0
- Outcome: judge rubric (grounded, complete, cites expected), reasoning first
- Efficiency: steps, tokens, cost, latency per task
- Dataset: real traffic + failures + attacks + synthetic; per-category results
- Regression suite in CI; stub write tools

GUARDRAILS CHECKLIST

- Threats: direct / indirect injection, jailbreak, leakage, multi-agent exploitation, tool misuse, memory poisoning, exhaustion
- Threat-model with MAESTRO (7 layers); check OWASP Agentic Top 10
- Policy layer before every tool call (`wrap_tool_call`)
- Input/output validation (`PIIMiddleware, LangChain4j guardrails, SafeGuardAdvisor`)
- Sandbox code (`DockerExecutionPolicy`)
- Budgets: `ToolCallLimitMiddleware, ModelCallLimitMiddleware, Spring AI tool-call limits`
- Audit trail · kill switch (default off for writes)

FRAMEWORK LANDSCAPE (2026-09-28)

Framework	Lang	Paradigm	State	Multi-agent	MCP/A2A	Status
LangGraph	Py, TS	graph over typed state	checkpoints, stores	tools, swarm, subgraphs	adapters / SDKs	1.2.12
LangChain create_agent	Py, TS	prebuilt agent + middleware	LangGraph	sub-agents as tools	adapters / SDKs	1.4.2
OpenAI Agents SDK	Py, TS	agents, handoffs, guardrails	sessions	handoffs, agents-as-tools	MCP / —	0.22.3
Claude Agent SDK	Py, TS	Claude Code harness	sessions	subagents	MCP / —	0.2.161
Microsoft Agent Framework	Py, .NET	agents + graph workflows	sessions, checkpoints	5 orchestrations	MCP / A2A	GA 1.0 (1.19.0)
CrewAI	Py	crews + flows	memory, @persist	sequential, hierarchical	MCP / A2A	1.15.22
smolagents	Py	code agents	in-run steps	managed agents	MCP / —	1.26.0
Google ADK	Py, Java, Go, TS	LLM + workflow agents	session, memory services	sub-agents	MCP / A2A	2.10.0
LangChain4j	Java	AI services + agentic	chat memory, scope	supervisor, planners	MCP / A2A client	1.20.2 (agentic beta)
Spring AI	Java	ChatClient + advisors	chat memory repos	tools, community subagents	MCP / community	2.0.1
Mastra	TS	agents + workflows	threads, semantic recall	agent networks	MCP / A2A	1.71.0
Rig	Rust	LLM building blocks	app-managed	composition	via rmcp / —	0.42.0

Deprecated: OpenAI Assistants API sunset 2026-08-26 → Responses + Conversations + Agents SDK · AutoGen / Semantic Kernel agents → Microsoft Agent Framework (GA 2026-04-03) · Prompt flow retires 2027-04-20 · LangChain `create_react_agent` → `create_agent` + middleware.